Thermal-Aware NoC Reconfiguration for MoE Inference in 3D DRAM-Stacked Systems

Anonymous Author(s)

***Abstract*—Three-dimensional DRAM stacking provides high memory bandwidth for Mixture-of-Experts (MoE) inference, but efficiently utilizing distributed compute and memory resources remains challenging. Input-dependent expert activation and concentrated memory accesses produce uneven communication demands that change across execution stages, causing contention on shared network paths. Thermal coupling between stacked logic and memory dies further limits sustained performance. Coarse-grained bandwidth configurations offer limited flexibility in allocating connectivity to changing heavy flows. We propose traffic- and thermal-aware NoC reconfiguration for MoE inference in 3D DRAM-stacked systems. Interconnected configurable switches establish dedicated paths for selected high-volume and long-range flows, reducing competition for router forwarding resources. A prediction mechanism estimates communication demand from expert activation and memory access behavior. Based on this demand, a configuration algorithm maximizes useful traffic assigned to the reconfigurable network under switch-conflict and shared-resource constraints. A timing check accounts for reconfiguration costs before applying an update. Coupled thermal prediction further guides task placement and regional voltage–frequency adjustment to alleviate cross-tier hotspots while limiting execution-time overhead.**

***Index Terms*—hybrid bonding, 3D integration, mixture-of-experts, network-on-chip, reconfigurable interconnect, memory-bank contention, thermal management**

I. Introduction

The rapid growth of AI models is increasing demands for compute throughput, memory capacity, and data-movement bandwidth, motivating three-dimensional (3D) integration of heterogeneous compute and memory resources. Commercial systems such as AMD's hybrid-bonded 3D V-Cache demonstrate the maturity of this approach, while Helios applies hybrid-bonded 3D DRAM to near-memory processing for large language model (LLM) serving [1], [2].

Deploying LLM inference on such 3D systems exposes three interrelated challenges. First, uneven accesses to model parameters and KV-cache data cause DRAM-bank conflicts,causing an increase in the delay of memory access time. Second, bursty and spatially skewed communication causes network-on-chip (NoC) contention when messages compete for the same links or router outputs. The busy resources can change between mixture-of-experts (MoE) model layers and communication phases, whereas a small set of predefined topologies offers limited ways to adapt local connections. Third, spatially aligned activity in the stacked Logic and DRAM tiers creates vertically coupled thermal hotspots, increasing peak temperature and the risk of thermal throttling.

Prior work addresses these challenges through expert placement, interconnect configuration, and thermal management. HiBand combines bank-local HB-DRAM, hierarchical data movement, and a small set of reconfigurable NoC modes [3]; Sem-MoE co-schedules expert placement and token routing to increase device-local expert activations [4]; and AMBER combines offline thermal-aware mapping with online subtask interleaving and DVFS [5].

Our experiments identify three remaining bottlenecks in memory access, on-chip communication, and thermal management. Concurrent Qwen3 requests contend for shared DRAM banks, delaying memory accesses. During Qwen3 prefill, uneven traffic creates queues along shared network paths. In the thermal study, AMBER-style management reduces throughput relative to the reference without thermal control, while hotspots remain aligned across the logic and DRAM tiers. These observations motivate examining how memory accesses and NPU-to-NPU messages compete for shared resources and how activity in the stacked tiers contributes to cross-tier hotspots.

To address the above challenges, we distribute concurrent memory accesses across DRAM banks and configure dedicated NoC paths for selected flows in each MoE communication phase to reduce queueing at congested links and router outputs. We also use coupled logic–DRAM thermal predictions to guide task placement and regional voltage and frequency adjustments, while accounting for data-copy and activation-transfer costs.

The contributions of the paper are as follows:

1) We identify phase-dependent NoC congestion in MoE inference on 3D DRAM-stacked systems. Input-dependent expert activation and concentrated memory accesses create uneven communication demand, while coarse-grained NoC configurations limit the ability to redirect traffic away from congested links and router outputs. We also observe persistent cross-tier hotspots under thermal management.
2) We propose a framework that jointly optimizes NoC reconfiguration, DRAM-bank placement, and thermal-aware execution. It configures dedicated paths for predicted heavy flows under physical port and path-conflict constraints, distributes concurrent accesses across DRAM banks, and uses coupled logic–DRAM thermal predictions to guide task placement and regional DVFS. These mechanisms aim to reduce network queueing and bank contention while mitigating cross-tier hotspots.
3) Across the evaluated MoE workloads, the proposed framework reduces mean NoC queueing latency by

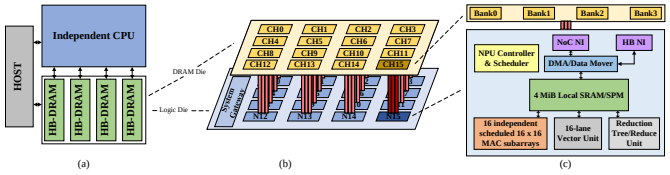


Fig. 1. Target 3D system. (a) Host and HB-DRAM organization. (b) Face-to-face integration of the 4×4 Logic-tier NPU mesh and the aligned DRAM channels. (c) Microarchitecture of an NPU–memory pair.

xxx% relative to the baseline. The estimated end-to-end inference latency decreases by xxx%, and the peak stack temperature decreases by xxx °C under matched thermal conditions.

II. Motivation

A. Target System and Workloads

Figure 1 shows a 3D accelerator in which the logic and memory tiers are integrated by face-to-face hybrid bonding. Each NPU cluster in the logic tier is vertically aligned with a DRAM partition and accesses it through a local hybrid-bond link. A horizontal 4×4 mesh NoC connects the NPU clusters and carries transfers between them, including MoE dispatch and combine traffic. Local memory accesses and inter-cluster transfers therefore use distinct physical paths. The vertical alignment also couples the temperatures of the logic and DRAM tiers, so concentrated activity in one tier can heat the other.

The main MoE evaluation covers Qwen3-30B-A3B [6] and Qwen1.5-MoE-A2.7B [7] for language-model inference, mini-SWE-agent [8] with a Qwen3-30B-A3B backend for SWE-bench Lite tasks [9], and LingBot TI2V [10] for video generation. The auxiliary thermal scenario combines WHISPER [11], an ORB-SLAM3-derived SLAM workload [12], and $\pi_{0.5}$ [13].

B. Phase-Dependent Congestion in MoE Communication

Problem. HiBand adapts communication locality by switching among four topologies— Full-Local, Local-Biased, Global-Biased, and Full-Global—that repartition bandwidth according to broad communication locality [3]. MoE communication can concentrate transfers around a small set of busy NPU nodes and links. When several messages need the same link or router output, they must wait even if other parts of the network have spare capacity. As hotspot locations change, the four predefined topologies may still leave concentrated flows competing for the same links and router outputs, causing persistent queueing despite spare capacity elsewhere.

Figure 2 shows the communication latency of Qwen3-30B-A3B prefill. As input length increases from 128 to 1024 tokens per request, the queueing share of mean network round-trip latency rises from 57.2% to 88.8%, while mean zero-load latency increases by 17.3%. Zero-load latency includes serialization and traversal without competing traffic; queueing is the additional delay under concurrent traffic. Thus, queueing at shared network resources dominates the growth in communication latency as input length increases.

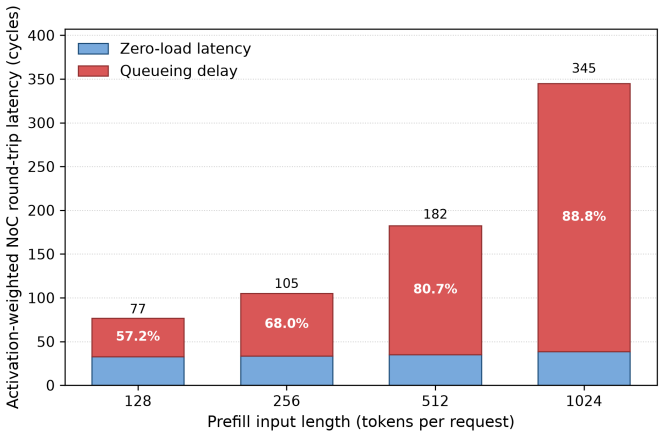


Fig. 2. Qwen3-30B-A3B prefill communication on the fixed Mesh. Each bar reports the activation-weighted mean NoC round-trip latency at one input length. Zero-load latency contains serialization and uncongested traversal; queueing is the additional delay in the concurrent replay of the same message set. From 128 to 1024 input tokens, the queueing share rises from 57.2% to 88.8%.

Root Cause. The gating network (Gate) selects a small set of experts for each token. During Dispatch, token activations are sent to the NPU nodes holding those experts; Combine returns the corresponding expert outputs. We define a flow as the messages from one source NPU to one destination NPU within a single communication phase. During Dispatch, when many tokens select the same experts, or several frequently selected experts reside on the same NPU, transfers concentrate at a small set of destinations. Flows whose routes overlap then compete for shared links and router outputs, creating queueing hotspots. Thus, hotspot formation depends on both the distribution of Gate selections and expert placement. Our controlled Qwen3 decoding test also shows that concentrated accesses can serialize requests at shared DRAM banks. Spreading those accesses across banks addresses memory-side waiting, while overlapping NoC routes still require path-level control.

Figure 3 maps these messages onto deterministic XY paths. Across MoE model layers 1, 24, and 48, the largest path-derived router demand occurs at R9, R5, and R10, respectively. This spatial concentration can force multiple flows to share a router output or link. Different concentrations can require different local connections even when their total volume and aggregate communication locality are similar. The design requirement is therefore to provide selectable local paths around the contested resources, with explicit capacity at their injection and receiving interfaces.

Opportunity: Configurable Local Connections. We place configurable switches (CSs) between neighboring NPU nodes in the same Logic tier. Each switch selects connections between four fixed corner ports. Using predicted hotspot locations, the controller configures nearby switches to redirect selected transfers around congested links and router outputs, and adjusts these connections as hotspot locations change. A selected connection takes data from an auxiliary router output and delivers it directly to the destination network interface

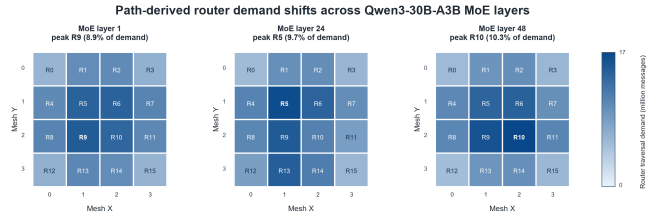


Fig. 3. Path-derived router demand under the same Qwen3 trace. Each heatmap aggregates exact source–destination messages over every router on their deterministic XY path. With one shared scale, the peak moves R9→R5→R10 across MoE layers 1, 24, and 48, exposing a time-varying spatial demand pattern.

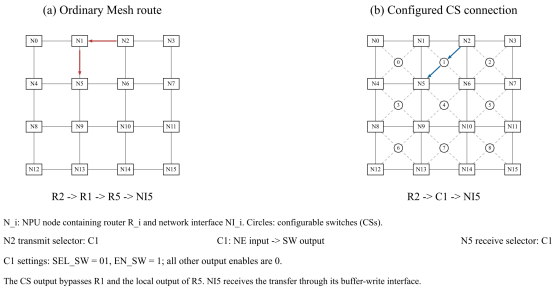


Fig. 4. Connection example for the NoC opportunity. (a) The ordinary N2-to-N5 route uses R1 and R5's local output. (b) A configured C1 connection delivers the transfer directly to NI5. The source transmit selector, CS connection, and destination receive selector are configured together.

(NI), bypassing intermediate Mesh routing and the destination router's ordinary local output along that connection. Traffic without a selected CS connection continues to use the Mesh.

Figure 4 illustrates the mechanism. The XY route from N2 to N5 traverses R2, R1, and R5. Switch C1 connects N1, N2, N5, and N6; setting its northeast input to its southwest output permits the alternative R2–C1–NI5 path. N2's transmit selector and N5's receive selector must both select C1. This connection removes that transfer from the R1 path and R5 local output and delivers the data to NI5's buffer-write interface. When other messages compete for R1's output toward R5 or R5's local output, diverting this transfer through C1 reduces the traffic using those outputs and can relieve queueing.

Concentrating activity in a small region also affects the temperature of both tiers. In our auxiliary thermal study, aligned logic and DRAM hotspots persisted under AMBER-style thermal management [5]. This motivates using coupled temperature predictions to guide task placement and regional DVFS.

III. Design

A. Reconfigurable Network

Figure 5(a) shows the reconfigurable network alongside the basic 4×4 Mesh. The logic tier contains sixteen NPU nodes, nine lower-level configurable switches (CSs), and four upper-level CSs. Nodes N_0–N_{15} and lower CSs CS_0–CS_8 are indexed in row-major order. Each lower CS physically connects to the NIs of the four corner nodes of a 2×2 group. A corner, edge, or interior node therefore connects to one, two, or four lower CSs, respectively. The upper CSs form a 2×2 network: CS_9, CS_{10}, CS_{11}, and CS_{12} connect down to CS_0, CS_2, CS_6, and CS_8, respectively. Each upper CS connects to its corresponding lower CS and two upper neighbors. For a local transfer, a lower CS selects one node-facing input and a different node-facing output. A connection between a node and the upper network instead selects one node-facing direction and U. The upper network provides additional paths between the four lower CSs attached to it. Upper CSs reach NPU nodes through these lower CSs. The two CS levels are connectivity levels within the same logic die. Their MUXes, configuration registers, and control gates are implemented as logic cells, with the connections routed in the die's metal stack. The separated planes in the figure depict this connectivity hierarchy.

Figure 5(b) shows the two CS circuits. A lower CS has four corner directions, northwest (NW), northeast (NE), southwest (SW), and southeast (SE), and an upper direction U. Only the four lower CSs listed above have a physical U attachment. In the other five instances, the U input data and valid signals are tied to zero, the U output enable is hardwired to zero, and the U-side ready input is tied to zero. These constant connections are fixed at implementation time and cannot be changed by configuration writes. Figure 5(b) depicts the lower-CS circuit with a connected U direction. An upper CS has two in-plane directions and a downward direction D. The lower and upper circuits use five 4:1 multiplexers (MUXes) and three 2:1 MUXes, respectively. Each output MUX is wired to the other directions' inputs, excluding a return to its own direction. The configuration register supplies an output-direction code, an input-selection code, and a connection enable (EN). Direction decoding enables at most one output, so each CS establishes at most one directed input–output connection.

A short transfer between two nodes attached to the same lower CS can use that CS directly between its source and destination NIs. This circuit provides an additional data channel alongside the Mesh, increasing the available path bandwidth and moving part of the traffic away from contended Mesh links and router outputs. Circuits through the upper network connect nodes attached to different U-connected lower CSs, bypassing intermediate Mesh routers. Multiple CS attachments allow a node to participate in separate circuits. A heavily communicating neighboring pair can use an additional CS path alongside its Mesh connection. These connections can be reassigned as the busy endpoints and node pairs change between communication phases. Traffic without an assigned circuit uses the ordinary Mesh. The aggregate transfer rate is also limited by NI service and SRAM bank bandwidth.

B. Node and Network Interface

Figure 6(a) expands an NPU node. Its NPU cluster contains a multiply–accumulate (MAC) array and a vector unit. A banked SRAM scratchpad holds local weights and activations. The SRAM access controller arbitrates bank requests from the NPU and the network interface (NI). The Mesh router retains

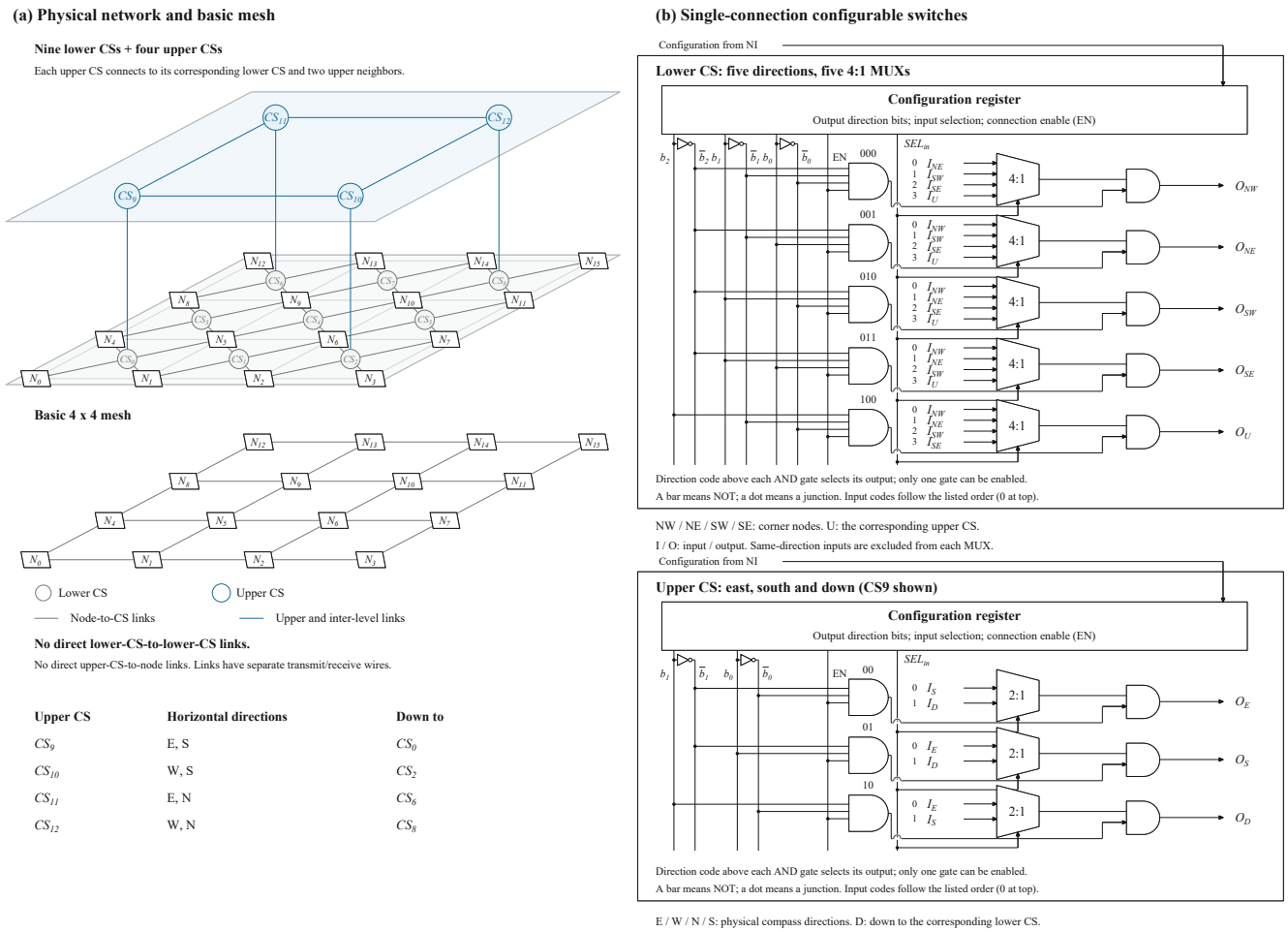


Fig. 5. Reconfigurable network and switch circuits. (a) Nine lower CSs and four interconnected upper CSs, with the basic Mesh shown for comparison. The separated planes denote connectivity levels on one logic die. (b) A U-connected lower CS and an upper CS. Each MUX excludes its own direction's input; output decoding enables one connection.

its four directional connections and a local NI connection. The NI also has separate transmit and receive connections to each adjacent lower CS. NW, NE, SW, and SE beneath the CSs in this figure identify their positions relative to the node; a CS northwest of the node uses its own SE direction to reach that node.

Figure 6(b) shows the NI data path. The transfer controller receives descriptors specifying the source and destination nodes, SRAM addresses, transfer length, packet-to-CS assignments, and the configuration version under which those attachments are usable. A direct memory access (DMA) read engine fetches source data and forms packets using the NI's configured payload size. The final packet carries the remaining bytes. The sending and receiving NIs use the same payload-size setting for a transfer. Transmit queues select the Mesh or an assigned CS circuit for each complete packet. A transfer may use several paths across its packets; the flits within one packet follow the same path. Each incoming data connection has a finite receive queue. Receive arbitration feeds the reassembly buffer, while packet-type decoding separates configuration commands from data. CS streams carry data and a valid indication forward, with a ready indication returning along the selected path. A transfer occurs when valid and ready are both asserted.

Each data packet identifies its transfer by `TransferID`, formed from the source NI and a source-local sequence number. `Offset` gives its byte position within the transfer, and `Length` gives the total transfer size. Receive descriptors associate a transfer with its destination SRAM base address. The NI derives each packet's index and expected payload length from its offset, the transfer length, and the agreed payload size. It checks the transfer identifier, offset alignment, and payload length before writing the payload to the base address plus `Offset`. A receipt bitmap has one bit per expected packet, updated after the SRAM accepts that packet's payload write; already-received packets are identified by their set bits. Transfers are admitted after allocating their receive descriptors and bitmap storage. Payloads can be written in arrival order while their offsets restore the destination data layout. The NI reports completion after all expected packet

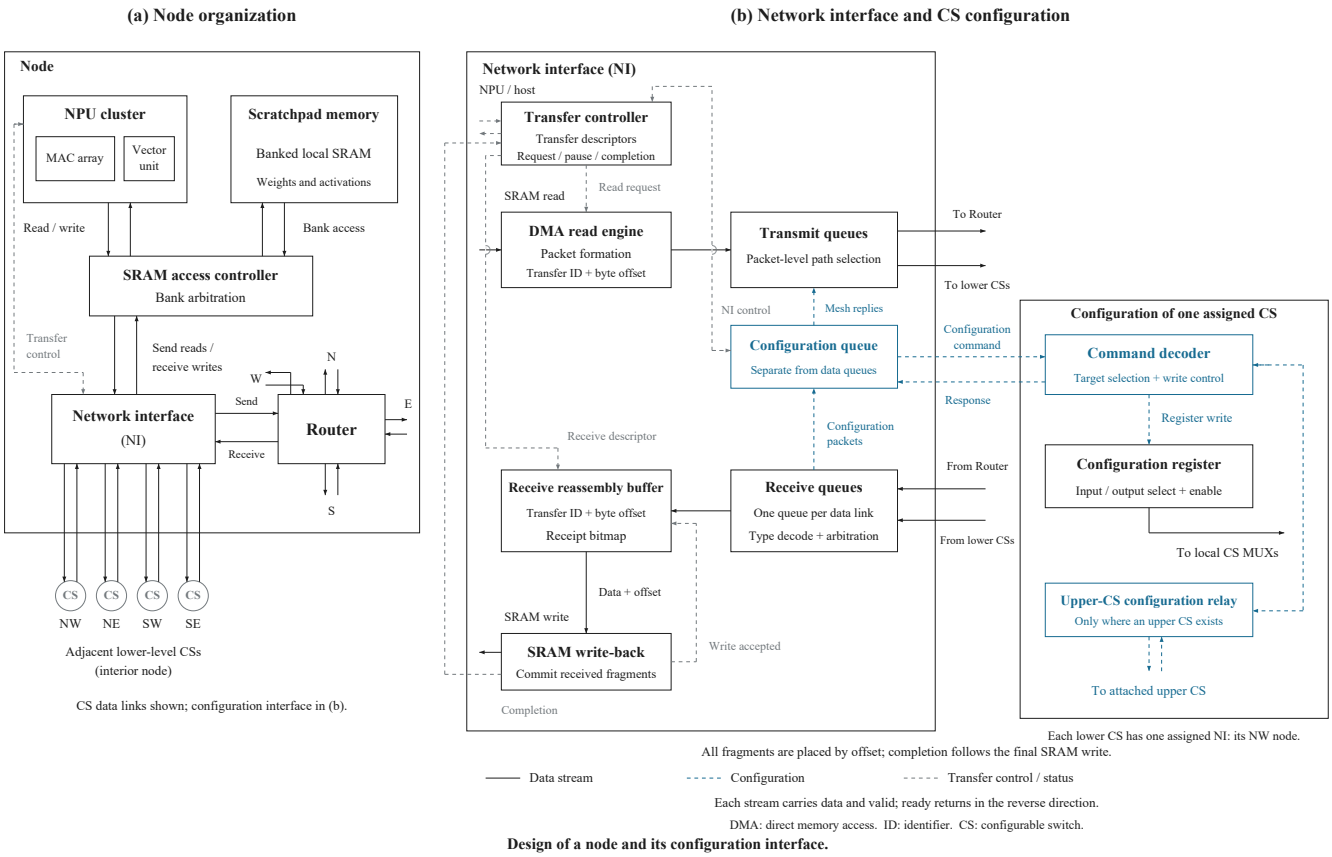


Fig. 6. Node and NI organization. (a) NPU, scratchpad, NI, Mesh router, and adjacent lower-CS data connections. (b) NI transmit, receive, and reassembly paths, together with the configuration interface of one assigned CS. Each lower CS is configured by its northwest node's NI.

writes have completed; the NPU then consumes the assembled data. This permits bounded receive staging without holding later fragments until earlier ones arrive. Reassembly writes and NPU accesses share the SRAM bank arbiter.

C. Traffic Prediction

The host CPU prepares circuit candidates before a communication phase and checks them when its current plan becomes visible. For Dispatch, we adapt the adjacent-layer expert-load predictor used by MixNet [14]. Let $\mathbf{x}_{\ell-1,s}$ denote the normalized expert-slot distribution of the preceding layer for tokens whose current source NI s is already known. Their predicted distribution is $\widehat{\mathbf{y}}_{\ell,s} = P_\ell \mathbf{x}_{\ell-1,s}$. The layer-specific matrix is fitted on development requests as

$$\min_{P_\ell \geq 0} \sum_i w_i \|\mathbf{y}_i - P_\ell \mathbf{x}_i\|_2^2, \qquad \mathbf{1}^\mathsf{T} P_\ell = \mathbf{1}^\mathsf{T}, \tag{1}$$

where $\mathbf{x}_i$ and $\mathbf{y}_i$ describe the same token group in adjacent layers, and w_i weights its valid slot count and recency. Matrix P_ℓ has E_ℓ rows and $E_{\ell-1}$ columns, where E_ℓ is the expert count of layer ℓ. Prefill and decode use separate models. A same-layer historical mean supplies the estimate when preceding-layer information is unavailable. Known source groups, expert placement, and packing rules convert these load estimates into source–destination demand; unknown sources remain unlocated. Once Top-K and packing metadata are visible, actual packet sizes replace the estimates. Source-read and transmit readiness determine packet release times.

For Combine, the confirmed Dispatch assignments identify the returning destinations. Output shape, precision, aggregation, and packing determine the return packets; expert completion determines their release windows. Demand history is indexed by layer, phase, prefill/decode, batch range, and mapping. Measured congestion additionally uses a fingerprint of the physical topology and effective CS configuration. Completed observations before the prediction cutoff supply history; a fully observed empty hotspot set differs from a missing observation. Historical hotspots guide candidate locations while current demand determines the decision.

D. Reconfiguration Problem

a) Inputs: For one configuration decision, $\mathcal{F}$ is the set of directed source–destination flows and $\mathcal{M}_f$ the remaining, unadmitted packets of flow f. Packet $m \in \mathcal{M} = \bigcup_f \mathcal{M}_f$ carries b_m useful payload bytes, and $D_f = \sum_{m \in \mathcal{M}_f} b_m$ is the flow's remaining volume. The wired topology supplies a set $\mathcal{P}_m$ of simple CS paths between that packet's source and destination; $\mathcal{P} = \bigcup_m \mathcal{P}_m$. Each path records its NI attachments and ordered CS input–output connections. The

baseline Mesh route is denoted by 0. A fixed planning interval of H NoC cycles and the visible state determine the available service budgets.

b) Decision variables and objective: Binary z_p selects CS path p. Binary $x_{m,p}$ assigns the whole packet m to that path. We maximize the useful volume assigned to the reconfigurable network:

$$\max_{\mathbf{z},\mathbf{x}} \quad V(\mathbf{z},\mathbf{x}) = \sum_{m\in\mathcal{M}} b_m \sum_{p\in\mathcal{P}_m} x_{m,p}. \tag{2}$$

Headers consume service but contribute no useful payload to V. Packets without a CS assignment use the Mesh. A flow can therefore use several CS paths and the Mesh across its packets.

c) Assignment and switch constraints: Let g_m indicate that the transfer is eligible for new CS admission: its source payload is resident, its receive context is allocated, and it has no existing path binding. The variable domains and assignment constraints are

$$z_p \in \{0,1\}, \qquad p \in \mathcal{P}, \tag{3}$$

$$x_{m,p} \in \{0,1\}, \qquad m \in \mathcal{M},\ p \in \mathcal{P}_m,$$

$$\sum_{p\in\mathcal{P}_m} x_{m,p} \le 1, \qquad m \in \mathcal{M}, \tag{4}$$

$$x_{m,p} \le z_p, \quad x_{m,p} \le g_m. \tag{5}$$

Equation (4) counts each packet once. Let $A_{c,p} = 1$ when path p traverses CS c, and zero otherwise. The single-connection CS circuit requires

$$\sum_{p\in\mathcal{P}} A_{c,p} z_p \le 1, \qquad c \in \{0,\dots,12\}. \tag{6}$$

The physical path table admits wired directions only, excludes a direction's return to itself and repeated CSs, and terminates at the specified destination. Source and destination attachments are actual NI links; nodes never act as intermediate CS forwarding points.

d) Shared service constraints: Resources $r \in \mathcal{R}$ include directed links, NI transmit and receive service, DMA engines, and SRAM-bank access resources. The coefficient $u_{m,p,r}$ is packet m's service work on resource r along path p, expressed in NoC-cycle equivalents using the resource's service rate. It includes the actual packet framing; $u_{m,0,r}$ is the corresponding Mesh cost. Budget C_r is the service available during H after reserving work for existing bindings, background traffic, and control messages outside $\mathcal{M}$. Using the derived Mesh assignment $x_{m,0} = 1 - \sum_{p\in\mathcal{P}_m} x_{m,p}$, the constraint is

$$\sum_{m\in\mathcal{M}} \sum_{p\in\mathcal{P}_m\cup\{0\}} u_{m,p,r} x_{m,p} \le C_r, \qquad r \in \mathcal{R}. \tag{7}$$

The $p = 0$ contribution retains the Mesh work of unassigned packets. For a shared source or destination resource, both route types consume its service. Different CS attachments may operate concurrently while sharing these budgets. Queueing, release dependencies, and finite-buffer backpressure enter the temporal check in the configuration algorithm.

e) Mapping variables to hardware: For the wired directions $i \neq o$ of CS c, constant $A_{c,i,o,p}$ indicates that path p uses input i and output o. The connection and enable are determined by $\mathbf{z}$:

$$h_{c,i,o}(\mathbf{z}) = \sum_{p\in\mathcal{P}} A_{c,i,o,p} z_p, \qquad e_c = \sum_{i\neq o} h_{c,i,o}(\mathbf{z}). \tag{8}$$

Equation (6) leaves at most one active pair. That pair determines the output code and the input's position in that output MUX's allowed-input list. When $e_c = 0$, the CS is disabled. The $x_{m,p}$ assignments identify the source-CS attachment for each packet in the NI transmit queue. These CS states and NI assignments form configuration $S(\mathbf{z},\mathbf{x})$, using the register fields and commands in Section III-F.

E. Configuration Algorithm

a) Path preparation: Offline, we enumerate local paths through a shared lower CS and cross-region paths through U-connected lower CSs and either simple direction around the upper network. Each path stores its direction codes, source and destination attachments, and resource coefficients. Before Top-K, predicted demand prepares a tagged candidate cache. Current packet sizes, eligibility, mapping, and configuration replace the cached assumptions when the plan becomes visible.

b) Flow ordering and allocation: Flows are ordered by decreasing remaining payload D_f; a fixed flow identifier breaks ties. Flows without eligible packets are skipped. Packets within a flow follow release order, then transfer identifier and byte offset. For a proposed circuit set, ASSIGN considers permitted CS paths in increasing unloaded latency and path-identifier order. It replaces each accepted packet's Mesh work with its CS work and checks the added service against the remaining budgets. Packets that cannot use a selected path remain on the Mesh. An unresolved Mesh budget excess can be reduced during construction; only complete vectors satisfying (3)–(7) enter the final candidate pool.

c) Conflict resolution: Starting from an empty proposed circuit set, each round considers the K_f largest remaining flows and at most K_p compatible paths per flow. These are CPU search budgets. EXTENSIONS forms single-path and mutually compatible two-path additions. Each addition preserves existing circuit choices and is assigned jointly; two separate single-path volumes are never added as a substitute for this allocation. The extension with the largest increase in V becomes the next construction state. Ties favor fewer changes from the active CS states, then fixed path identifiers. Construction stops when no volume can be added, no compatible path remains, or its time budget expires. Because each successful round reserves a new CS, there are at most 13 rounds. Thus a two-path extension budget permits configurations containing more than two circuits. FEASIBLE stores the tested allocations that satisfy all constraints; the active configuration S_0 is also retained.

d) Reconfiguration check and application: The feasible pool is checked in decreasing V. A tie with the active configuration retains it; other ties favor fewer state changes. A

Algorithm 1 Volume-based NoC configuration

Require: Current packets $\mathcal{M}$, budgets C_r, cache, active S_0
Ensure: Retain S_0 or apply a checked configuration
Sort flows by D_f and packets by release order
$S \leftarrow \varnothing$; $\mathcal{C} \leftarrow \{S_0\}$
while construction budget remains **do**
  $\mathcal{E} \leftarrow \textsc{Extensions}(S, K_f, K_p)$
    ▷ One or two new CS-disjoint paths
  for $S' \in \mathcal{E}$ **do**
    $(\mathbf{z}', \mathbf{x}') \leftarrow \textsc{Assign}(S', \mathcal{M})$
    if $\textsc{Feasible}(\mathbf{z}', \mathbf{x}')$ **then**
      Add $S'(\mathbf{z}', \mathbf{x}')$ to $\mathcal{C}$
    end if
  end for
  Choose the extension with largest added V
  if no positive increase **then**
    break
  end if
  $S \leftarrow$ chosen extension
end while
Order $\mathcal{C}$ by V, then fewer CS changes
for $S' \in \mathcal{C}$ in this order **do**
  if $S' = S_0$ or decision budget expired **then**
    return S_0
  end if
  Refresh state; check completion and background delay
    ▷ Control and shared-endpoint costs included
  if versions, eligibility, and timing checks pass **then**
    Pause new admissions and finish old transfers
    Collect drain responses; write CS states and confirm
    Install packet assignments and version; confirm
    Resume admissions; **return** S'
  end if
end for
return S_0

time-window model propagates serviced packets through the selected routes, source reads, receive queues, and destination bank writes. It includes the candidate's telemetry, CPU computation, pause, drain, writes, descriptor installation, and resume on a common timeline. Let $\widehat{T}(S)$ be the latest estimated completion of the current wave's required consumers, and $\eta(S)$ a paired-error margin from independent calibration requests. A proposed update is accepted only if

$$\widehat{T}(S) + \eta(S) < \widehat{T}(S_0). \tag{9}$$

For background wave b, its estimated completion $\widehat{T}_b$ also satisfies $\widehat{T}_b(S) - \widehat{T}_b(S_0) + \eta_b(S) \leq \epsilon_b$, where η_b is its error margin and ϵ_b its allowed delay. These checks control application of the volume-based allocation. The active network continues during computation. Before applying an update, the CPU refreshes completed and bound work and rechecks versions, eligibility, and the timing conditions. If no candidate passes within the decision budget, it retains S_0.

Algorithm 1 gives the steps. The NI protocol finishes old transfers before changing CS states, installs the packet assignments with the matching version, and resumes after all required acknowledgements. Completed observations update demand and configuration-specific congestion histories separately.

TABLE I
CONFIGURATION PACKET AND COMMAND FIELDS.

Field	Meaning
Type	Distinguishes configuration commands, responses, and data.
Destination	Mesh destination NI assigned to the target.
TransactionID	Matches a request to its response.
ConfigVersion	Identifies the CS image and matching NI assignments.
Target	CS index c, or transfer identifier for NI descriptor installation.
Operation	CS write, NI descriptor installation, pause, status, or resume.
State	CS tuple q_c, or NI packet-to-CS assignments indexed by byte offset.
Status	Completion or failure reported in a response.

F. Configuration Interface

The host CPU sends configuration packets over the ordinary Mesh to the NI assigned to the target CS. Each lower CS is assigned to the NI of its northwest corner node. This fixes one configuration writer per CS, independently of its four neighboring data connections. The NI places commands in a separate configuration queue and sends them through a local command/response interface to the CS-side configurator. The NI serializes writes to each assigned CS and rejects obsolete configuration versions before forwarding commands. The CS-side command decoder acts as the configurator: it decodes the target and operation, writes the CS configuration register, and returns a response. For an upper CS, the corresponding lower CS relays the command and response over a fixed inter-level control interface. These local control interfaces operate independently of the CS data MUX settings. Responses return through the NI and ordinary Mesh.

Table I defines the packet and command fields. A CS state is $q_c = (s_c^{\text{out}}, s_c^{\text{in}}, e_c)$, where $c \in \{0, \ldots, 12\}$ identifies the target CS, s_c^{out} selects its output direction, s_c^{in} selects an allowed input at that output's MUX, and e_c enables the connection. The lower CS uses three output bits, two input bits, and one enable bit; the upper CS uses two output bits, one input bit, and one enable bit. Lower output codes 000–100 select NW, NE, SW, SE, and U, respectively. Upper codes 00, 01, and 10 select the instance's horizontal direction, vertical in-plane direction, and D. Input codes follow the remaining input order shown at each MUX. A cleared enable or an unused output code disables the connection.

The response echoes `TransactionID`, `Target`, and `ConfigVersion`; an upper-CS write is acknowledged after the upper register accepts it. The NI handles its own pause, status, and resume commands locally. They control transfer admission and completion reporting, while CS-targeted commands go to the CS-side configurator. NI descriptor-installation commands associate a transfer identifier with its packet-to-CS assignments and configuration version. Each assigned packet is identified by its byte offset; its source-CS attachment is stored as a route tag in the transmit queue. Packets without such an assignment use the Mesh. The transfer

controller acknowledges installation after storing the descriptor and confirming the required receive context. Its version must match the applied configuration before CS admission.

To change the network, the host first pauses admission of new CS transfers at all NIs. Previously admitted transfers finish transmission, reassembly, and SRAM write-back. The host collects their drain responses over the Mesh, writes the new CS states, and waits for every target's write response. It then installs the matching NI descriptors and waits for their acknowledgements before resuming admission. A failed or incomplete update keeps CS transmission paused. This sequence prevents data from traversing a partially updated circuit.

IV. Evaluation Results and Analysis

A. Experimental Setup

We evaluate the proposed design on a face-to-face hybrid-bonded 3D accelerator. Table II lists the common platform parameters. The evaluation uses 32 GiB of HB-DRAM. Each comparison starts from the same input and initial system state and uses the same precision, resource budget, and workload completion criteria. We account for each method's additional hardware and runtime costs separately.

Table III lists four main MoE workloads and one auxiliary thermal scenario. We freeze the input manifests, arrival order, seeds, and stopping rules before collecting performance results.

For each experimental line, we first run the complete workload under both configurations and record stage latencies and the end-to-end critical path. Subsequent paired points measure the same one or two MoE layers with identical input and routing samples. We extrapolate both arms using the same layer selection, weights, and dependency model, including non-MoE stages and control costs. Recomputing the critical path preserves overlap among compute, memory, and communication. We validate the estimates against the complete runs and report measured layer latency, estimated end-to-end latency, and estimated speedup separately. The corresponding estimation errors are xxx. All reported execution times refer to the modeled target system.

B. End-to-End Performance Comparison

The primary comparison pairs the common platform baseline B with B plus the full design. We separately compare with prior methods that pass the same workload and timing checks under matched resource constraints. The HiBand reference implementation and other candidate NoCs enter this comparison after qualification. Table IV records the scope of each comparison.

Figure 7 compares completion times for each main workload. For each pair, we compute the latency reduction as $1-T_{\mathrm{design}}/T_{\mathrm{ref}}$, where T_{ref} is the completion time of B or the named external comparator. We also report time to first token and time per output token for the language-model workloads. For mini-SWE-agent, we report model-call latency and task latency separately and record task outcomes. For LingBot, we report the latency of a complete generation. The measured end-to-end latencies, paired reductions, and workload-level exceptions are xxx.

TABLE II
Common hardware and simulation settings.

Component	Configuration
Logic tier	16 NPU clusters at 1 GHz
NPU cluster	4096 MACs, 16-lane vector unit, 4 MiB SRAM
Base NoC	4×4 mesh, 256-bit links at 1 GHz
HB-DRAM	32 GiB, 16 partitions of 2 GiB each, 16 logical banks per partition
Vertical link	25.6 GB/s per NPU–memory pair
Host	Four CPU cores, PCIe 5.0, host memory xxx
Simulation	Compute, memory, and NoC models xxx
Thermal model	Ambient and initial temperatures xxx, temperature limit xxx

C. Supporting Results and Thermal Ablation

a) Phase-dependent congestion and bank waiting.: To explain the latency differences in Figure 7, we trace matched MoE communication phases from memory requests through consumer readiness. We measure bank queueing before data becomes available, injection and router queueing, unloaded traversal, and synchronization at the slowest endpoint. We record configuration delay on the same timeline and attribute only exposed delays to the critical path. Comparing packet tails and router activity across phases tests whether a change in hotspot location translates into earlier consumer readiness. A comparison with fixed network policy and a bank-policy toggle can isolate the incremental effect of bank conflict mitigation. The paired changes in exposed waiting and packet tail latency are xxx.

b) Incremental effect of thermal management.: We compare NoC with bank conflict mitigation and thermal management against the same NoC and bank mechanisms with thermal management disabled. Both arms start with identical temperatures, inputs, and thermal constraints. We track temperatures over the complete execution timeline and account for prediction, throttling, redistribution, and recovery where they occur. We report natural and thermally stressed scenarios separately. Peak temperature, time above the threshold, and completion time are xxx.

D. Scalability and Sensitivity Analysis

We vary system scale and input scale separately. The system-scale sweep starts at 16 NPU clusters and holds workload fixed. At each scale, we report the topology, mapping, capacity, and bandwidth. The input-scale sweep fixes the platform and varies request concurrency or input size. We report absolute completion time and paired latency reduction at each point. To examine design sensitivity, we vary the local connection budget and reconfiguration interval one at a time and record completion time, packet tail latency, and exposed configuration cost. The tested ranges and observed trends are xxx.

TABLE III
WORKLOADS, INPUT MANIFESTS, AND MEASUREMENT BOUNDARIES.

Workload	Inputs and configuration	Measurement boundary
Qwen3-30B-A3B	Prompt source and IDs xxx, input/output lengths xxx, concurrency xxx	Prefill, decode, and complete request
Qwen1.5-MoE-A2.7B	Prompt source and IDs xxx, lengths xxx, TP/PP mapping xxx	Prefill, decode, and complete request
mini-SWE-agent + Qwen3	SWE-bench Lite instance IDs and split xxx, tool and stopping rules xxx	Model calls, full agent task, and task outcome
LingBot TI2V	Text/image IDs xxx, resolution xxx, frames xxx, steps and seeds xxx	Encoding, denoising, scheduler, and VAE decoding
WHISPER + SLAM + $\pi_{0.5}$	WHISPER version xxx, SLAM and $\pi_{0.5}$ traces xxx, request schedule xxx	Joint thermal scenario, execution coverage xxx

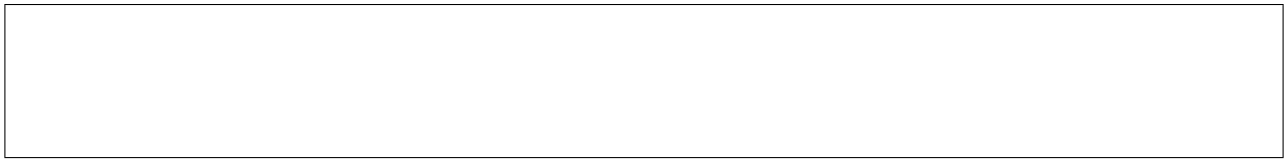


Fig. 7. End-to-end latency by workload for B, B plus the full design, and qualified external comparators. Each panel uses identical inputs and completion criteria. Labels distinguish complete-run measurements from estimates based on sampled MoE layers.

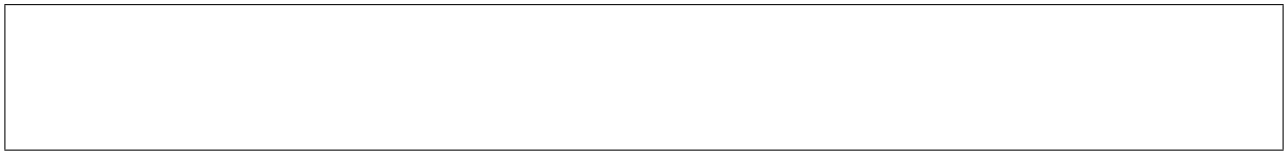


Fig. 8. Critical-path latency breakdown for matched MoE communication phases. The timeline shows bank service, data readiness, NoC injection and queueing, configuration, and consumer synchronization. Companion panels show packet tail latency and router activity using common scales across configurations.

TABLE IV
CONFIGURATIONS AND SCOPE OF THE PERFORMANCE COMPARISONS.

Configuration	Comparison scope
B	Common platform baseline, frozen configuration xxx
B + full design	Paired end-to-end comparison with B
HiBand reference	External comparison, qualification pending
Other NoC candidates	Configurations and qualification results xxx



Fig. 9. Temperature and workload progress with thermal management enabled and disabled. Both configurations use the same NoC and bank policies. Markers denote thermal triggers, throttling, and work redistribution where observed.

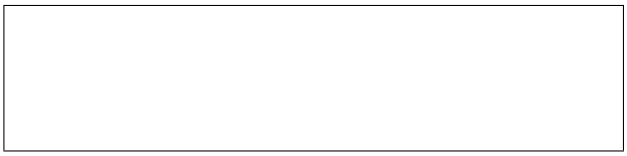


Fig. 10. Scalability and sensitivity. Panels show (a) system scale at fixed workload, (b) input scale at fixed system size, and (c) sensitivity to the local connection budget and reconfiguration interval. Markers indicate the default configuration and whether overall latency is measured or estimated.

TABLE V
HARDWARE IMPLEMENTATION COST BY EVALUATION STAGE.

Stage	Conditions and reported results
FPGA prototype	Device/tool/clock xxx, LUT/FF/BRAM/DSP xxx, achieved frequency xxx
ASIC synthesis	Process/library/PVT/clock xxx, area xxx, power estimate xxx
Post-route ASIC	Extraction/PVT/activity xxx, area xxx, timing slack xxx, dynamic and leakage power xxx

E. Hardware Cost and Runtime Overhead

We report implementation costs for local switches, links and configuration logic, bank control, and thermal control. Table V separates FPGA resource utilization, ASIC synthesis, and post-route results. Within each stage, baseline and enhanced implementations use the same tools and constraints. We state whether memories, physical interfaces, and cross-tier wiring are included and report module-level costs and system-level increments where available.

Table VI reports algorithm execution separately from the de-

TABLE VI
RUNTIME OVERHEAD OF ONLINE DECISION ALGORITHMS.

Algorithm	Time/call	Calls	Exposed share
Hotspot prediction	xxx	xxx	xxx
Connection selection	xxx	xxx	xxx
Bank policy	xxx	xxx	xxx
Thermal control	xxx	xxx	xxx

lay of applying its decisions. For each algorithm, we measure per-call time and call count, then report the fraction of total application latency exposed on the critical path. Configuration transfer, hardware reconfiguration, and thermal migration are accounted for as separate events.

V. Related Works

VI. Conclusion

References

[1] J. J. Wuu, R. Agarwal, M. Ciraula, C. Dietz, B. Johnson, D. Johnson, R. Schreiber, R. Swaminathan, W. Walker, and S. Naffziger, "3D V-Cache: The implementation of a hybrid-bonded 64mb stacked cache for a 7nm x86-64 CPU," in *2022 IEEE International Solid-State Circuits Conference*, 2022, pp. 428–429.

[2] C. Li, Y. Yin, C. Xue, Z. Wang, F. Bai, Y. Guo, X. Jiang, Q. Wu, Y. Xie, and G. Sun, "Helios: Enabling dynamic KV cache management for near-memory processing via hybrid-bonding-based 3D-DRAM," in *2026 59th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2026.

[3] H. Wang, Y. Wang, Y. Su, J. Yang, Y. Hu, and S. Yin, "HiBand: A hierarchical bandwidth-aware HB-DRAM accelerator with array-level co-execution and reconfigurable NoC for LLM inference," in *Proceedings of the 63rd ACM/IEEE Design Automation Conference*, 2026.

[4] Y. Li, Z. Zhang, Z. Wang, P. Chen, and P. Zheng, "Semantic parallelism: Redefining efficient MoE inference via model-data co-scheduling," in *International Conference on Learning Representations*, 2026. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2026/hash/f0552f14388d95b19740dee809f5cad1-Abstract-Conference.html

[5] X. Lin, S. Jia, H. Zhu, X. Jia, S. Liu, and C. Chen, "Amber: Thermal-aware spatio-temporal scheduling for edge llm inference on 3d dram-on-npu systems," in *2026 IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2026.

[6] A. Yang *et al.*, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[7] Qwen Team, "Qwen1.5-MoE: Matching 7b model performance with 1/3 activated parameters," 2024. [Online]. Available: https://qwenlm.github.io/blog/qwen-moe/

[8] SWE-agent Team, "mini-SWE-agent," 2025. [Online]. Available: https://github.com/SWE-agent/mini-swe-agent

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" 2023. [Online]. Available: https://arxiv.org/abs/2310.06770

[10] S. Ma, J. Liao, X. Wang *et al.*, "Scaling mixture-of-experts video pretraining for embodied intelligence," 2026. [Online]. Available: https://arxiv.org/abs/2607.07675

[11] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[12] C. Campos, R. Elvira, J. J. G. Rodriguez, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial and multi-map SLAM," 2020. [Online]. Available: https://arxiv.org/abs/2007.11898

[13] Physical Intelligence, K. Black, N. Brown *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," 2025. [Online]. Available: https://arxiv.org/abs/2504.16054

[14] X. Liao, Y. Sun, H. Tian *et al.*, "MixNet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training," in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025. [Online]. Available: https://cse.hkust.edu.hk/~kaichen/papers/mixnet-sigcomm25.pdf